

Leak-Free Dynamic Interaction Graphs Improve the Probabilistic Calibration of Graph-Based Multi-Agent Trajectory Prediction

Cristian Cubides

Research note on MTP-GO (Westny et al., IEEE T-IV 2023) — code: github.com/Cristiansch98/mtp-go

Abstract—MTP-GO combines a graph-neural-network encoder-decoder with neural-ODE motion models and is a strong baseline for probabilistic multi-agent trajectory prediction. We present a code-level analysis of its decoder and identify two structural weaknesses: (i) at every rollout step the decoder consumes the *ground-truth future interaction topology*, during training and evaluation, and (ii) the decoder GNN receives *no edge features at all*—inter-agent distances inform only the encoder. We propose two minimal, deployable modifications: *dynamic decoder edges*, which re-derive inter-agent distances at each step from the model’s own fed-back predictions and inject them through the same learnable Gaussian kernel used by the encoder, and a *presence-agnostic full topology* that removes the last ground-truth future input. We hypothesize, from the model’s covariance-propagation structure, that the change should primarily improve uncertainty calibration rather than point accuracy. Multi-seed experiments on the inD dataset confirm this: best validation NLL improves from -0.61 ± 0.59 to -3.02 ± 1.61 nats (better in 3/3 seeds), test average NLL from -0.095 to -0.173 , while ADE/FDE and miss rate are statistically unchanged, and removing the topology leak costs ≤ 0.04 m ADE. The result is a better-calibrated MTP-GO whose decoder needs no future information at inference time.

Index Terms—trajectory prediction, graph neural networks, mixture density networks, uncertainty calibration, neural ODEs.

I. INTRODUCTION

Forecasting the joint motion of road users is a core competence of autonomous driving systems, and the field has converged on graph-based encoders that model agent interaction explicitly. MTP-GO [1] is a representative and influential design: a GRU cell whose gates are graph neural networks (GNNs) encodes the observed scene, a second GRU-GNN decodes into the inputs of a differentially-constrained motion model (a neural ODE [4]), and an extended-Kalman-filter (EKF) style covariance propagation turns the rollout into a Gaussian mixture over future trajectories. The design couples strong point accuracy with the ability to report usable uncertainty, and its evaluation on the highD/rounD/inD drone datasets [3] made it a common baseline.

This note asks a simple question of the *decoder*: what does it actually know about the scene while it rolls the future out? A close reading of the reference implementation gives two surprising answers. First, the interaction graph used at future step $t+k$ is the *ground-truth future edge set*—which agents are present and connected at a time that has, from the model’s point of view, not happened yet. This information is consumed at every one of the 25 decoding steps, in training and in evaluation alike, so published metrics quietly benefit from an oracle that no deployed system has. Second, and in tension with the paper’s emphasis on interaction modeling, the decoder GNN is constructed *without an edge-feature channel*: the

precomputed future inter-agent distances are written to disk by preprocessing but never loaded by any model code. All decoder message passing is unweighted; only the encoder sees distances, through a Gaussian kernel with a learnable bandwidth.

We contribute: (1) a precise statement of both defects in the notation of the original paper (Sec. III); (2) two minimal fixes—*dynamic decoder edges* and a *presence-agnostic full topology*—that make the decoder self-contained and deployable (Sec. IV); (3) a mechanistic hypothesis for *why* the fix should improve probabilistic calibration specifically, derived from the covariance-propagation path of the architecture (Sec. IV-C); and (4) a multi-seed empirical test on inD that confirms the hypothesis—a large, seed-consistent NLL improvement at unchanged displacement error—reported together with its honest null results (Sec. V–VI).

II. MTP-GO PRELIMINARIES

A traffic scene with agents $\mathcal{V}$ is a sequence of graphs $\mathcal{G}_t = (\mathcal{V}_t, \mathcal{E}_t)$, complete over the agents present at time t . Node features are kinematic states; edges carry the Euclidean distance $d_{ij,t}$ between agents. The encoder is a GRU whose gates are GNN layers; its message passing weights each edge by a Gaussian kernel with learnable bandwidth β (initialized at 20 m),

$$w_{ij,t} = \exp(-d_{ij,t}^2/\beta^2) \quad (1)$$

so that near agents exchange strong messages and far agents are suppressed smoothly. From the final encoder state, mixture logits α_i define trajectory-level weights $\pi_{i,m}$ over M modes, and the model outputs, for agent i and horizon step k , the mixture density

$$p(\mathbf{y}_{i,k} | \mathcal{G}_{1:t}) = \sum_{m=1}^M \pi_{i,m} \mathcal{N}(\mathbf{y}_{i,k}; \hat{\mathbf{x}}_{i,m,k}, \mathbf{P}_{i,m,k}) \quad (2)$$

The means come from a motion model rather than a free-form head: a decoder GRU-GNN produces control inputs $\mathbf{u}_{i,m,k}$ for a second-order neural ODE,

$$\dot{\mathbf{p}}_i = \mathbf{v}_i, \quad \dot{\mathbf{v}}_i = f_{\theta}(\mathbf{x}_i, \mathbf{u}_i) \quad (3)$$

integrated with a fixed-step Runge–Kutta-4 scheme over each 0.2 s step. The covariances are propagated EKF-style along the rollout,

$$\mathbf{P}_{k+1} = \mathbf{F}_k \mathbf{P}_k \mathbf{F}_k^T + \mathbf{G}_k \mathbf{q}_k \mathbf{G}_k^T \quad (4)$$

where $\mathbf{F}_k, \mathbf{G}_k$ are Jacobians of the motion model and, crucially for what follows, the process noise $\mathbf{q}_k$ is *generated by the decoder GNN* at every step. Training minimizes the masked negative log-likelihood

$$\mathcal{L} = -\sum_{i,k} \log \sum_m \pi_{i,m} \mathcal{N}(\mathbf{y}_{i,k}; \hat{\mathbf{x}}_{i,m,k}, \mathbf{P}_{i,m,k}) \quad (5)$$

under an objective schedule (winner-takes-all warm-up before the NLL phase) with teacher forcing annealed from probability 0.2 to 0 over the first half of training—i.e., scheduled sampling [6] is already present in the reference code. Fed-back states are detached, so no gradients flow through the rollout recursion.

III. WHAT THE DECODER ACTUALLY SEES

Finding 1 (future-topology leak). At decoder step k the hidden state of agent i is updated by unweighted message passing over the *ground-truth* edge set of future time $t+k$:

$$\mathbf{h}_{i,k} = \Phi(\mathbf{h}_{i,k-1}, \{\mathbf{h}_{j,k-1} : (j,i) \in \mathcal{E}_{t+k}^{\text{gt}}\}) \quad (6)$$

The edge set $\mathcal{E}_{t+k}^{\text{gt}}$ encodes which agents exist in the scene at future step $t+k$ —information unavailable at inference in any deployed setting. Because graphs are complete over present agents, the leak is exactly *future presence*; it is used in training and evaluation alike, so reported benchmarks are internally consistent but not achievable by a causal system.

Finding 2 (edge-blind decoder). The decoder’s GRU-GNN cell is constructed with no edge-feature dimension. The distances that preprocessing computes for future frames are dead data, never loaded. Hence Φ in (6) aggregates all present neighbors with *equal weight*, regardless of whether they are 2 m or 80 m away, and this blindness persists for the whole 5 s horizon while the scene geometry evolves. The interaction graph the paper motivates is, during decoding, a presence indicator only.

Three consequences follow. First, a protocol–deployment mismatch: published numbers are conditioned on an oracle a real system cannot query, and nothing in the evaluation quantifies what the oracle is worth—it could be propping up accuracy, calibration, both, or neither. Second, a modeling inconsistency: the architecture’s central claim is that interaction should be modeled *throughout* the prediction, yet interaction strength is frozen at its encoded summary the moment decoding begins, while agents may close or open tens of meters of separation within the horizon. Third, the preprocessing pipeline computes and stores future inter-agent distances that no model code ever reads—suggesting a decoder edge-feature path was intended but never wired in. Our modification can be read as completing that apparent intention, causally.

IV. METHOD

A. Dynamic decoder edges

We give the decoder the same distance-kernel machinery as the encoder, driven purely by its own predictions. At step k , each agent’s fed-back per-mode positions $\hat{\mathbf{p}}_{i,m,k-1}$ are pooled with the (detached) mixture weights,

$$\bar{\mathbf{p}}_{i,k-1} = \sum_{m=1}^M \tilde{\pi}_{i,m} \hat{\mathbf{p}}_{i,m,k-1}, \quad \tilde{\pi}_i = \text{softmax}(\text{sg}[\alpha_i]) \quad (7)$$

where $\text{sg}[\cdot]$ is stop-gradient. Pairwise distances are recomputed from these pooled positions,

$$\hat{d}_{ij,k} = \|\bar{\mathbf{p}}_{i,k-1} - \bar{\mathbf{p}}_{j,k-1}\|_2 \quad (8)$$

and enter the decoder cell (now built with an edge channel) through its own learnable Gaussian kernel,

$$\hat{w}_{ij,k} = \exp(-\hat{d}_{ij,k}^2 / \beta_{\text{dec}}^2) \quad (9)$$

Two design decisions matter. *Causality*: distances at step k use positions from step $k-1$ —everything the model knows when predicting step k ; using the precomputed future distances would reintroduce a leak. *Gradient policy*: positions and mixture weights are detached, so the new path trains only the GNN weights and the bandwidth β_{dec} , mirroring the detached state feedback of the original architecture and leaving its optimization dynamics untouched. Under teacher forcing the fed-back positions are ground truth for past-of-current-step states, exactly as for the state feedback itself. Training throughput cost is $\approx 25\%$.

B. Presence-agnostic topology

To remove Finding 1 we replace $\mathcal{E}_{t+k}^{\text{gt}}$ with the complete graph (with self-loops) over all agents observed in the sample,

$$\tilde{\mathcal{E}} = \{(i,j) : i,j \in \mathcal{V}\} \quad (10)$$

generated on the fly from the node count (verified byte-identical to the preprocessing files on 100/100 test samples). The decoder then uses *no* ground-truth future input of any kind; combined with Sec. IV-A the decoder is fully deployable.

C. Hypotheses: why this should help, and where

H1 (calibration, main effect). The decoder GNN’s primary probabilistic output is the process noise $\mathbf{q}_k$ in (4): step-wise covariance growth is a *function of the decoder’s message passing*. Interaction-driven uncertainty—a conflict partner approaching, a leader braking—is precisely the signal carried by evolving inter-agent distance. A decoder that sees $\hat{d}_{ij,k}$ can modulate $\mathbf{q}_k$ scene-adaptively; an edge-blind decoder must amortize one average noise schedule over all scenes. We therefore expect the gain to appear in NLL (which (5) ties directly to $\mathbf{P}$) rather than in ADE/FDE.

H2 (phantom agents). With the presence-agnostic topology (10), agents that have left the scene (or not yet entered it) keep exchanging messages. Only a distance-dependent weight (9) can drive such phantom contributions to zero, since $\hat{w}_{ij,k} \rightarrow 0$ as $\hat{d}_{ij,k}$ grows. Hence dynamic edges should recover most of the calibration lost when the ground-truth presence oracle is removed.

H3 (point accuracy, expected null). The mean trajectory is dominated by the motion-model integration of (3) from the encoder’s summary; on inD, agent behavior over 5 s is largely determined by the observed kinematics. We expect displacement metrics to move little—and, correspondingly, that removing the presence oracle costs little ADE, i.e., that the leak was propping up calibration rather than accuracy.

V. EXPERIMENTS

Setup. We use the inD intersection dataset [3], recordings 18–29, preprocessed exactly as in [1]: 7,820 samples (6,159/741/920 train/val/test; the sample count matches the paper, confirming replication), 3 s observation and 5 s horizon at 5 Hz. All runs use the repository defaults of [1]: second-order neural-ODE motion model, hidden size 64, one GNN layer, $M=8$ mixtures, RK4, batch 128, Adam at 10^{-4} , 200 epochs, the original objective schedule and annealed teacher forcing. The baseline reproduces the published per-class accuracy of MTP-GO on inD (aggregate test ADE/FDE 1.006/2.835 m against published per-class ADE 0.42–1.27 m). Metrics are most-likely-component ADE/FDE, average per-step NLL of the position marginal (ANLL), miss rate (final error > 2 m, ≥ 4 s

horizon), and best validation NLL. Main comparisons use three seeds (1234, 1, 42) per variant on one RTX 5090 (≈ 66 min per baseline run).

Main result: calibration improves, seed-consistently. Table I compares the baseline decoder with dynamic edges (Sec. IV-A) over three seeds. Best validation NLL improves by 2.41 nats on average and in every seed; test ANLL improves in 2 of 3 seeds with a mean gain of 0.078 nats. Displacement metrics and miss rate are statistically unchanged: the paired differences ($+0.005$ m ADE, $+0.045$ m FDE) are an order of magnitude smaller than the seed spread of the baseline itself (ADE 0.983–1.049 m). This is exactly the H1/H3 pattern. With $n=3$ seeds the paired t-test on best validation NLL gives $t=-2.5$ ($p \approx 0.13$); we report direction-consistency rather than claiming formal significance.

TABLE I—Baseline vs. dynamic decoder edges on inD; mean $\pm$ std over seeds 1234/1/42 (identical splits). Bold: better mean.

Metric	Baseline	Dynamic edges
test ADE (m)	1.013 $\pm$ 0.034	1.018 $\pm$ 0.038
test FDE (m)	2.852 $\pm$ 0.118	2.897 $\pm$ 0.118
test ANLL (nats)	-0.095 ± 0.018	-0.173 $\pm$ 0.064
miss rate	0.418 $\pm$ 0.030	0.405 $\pm$ 0.027
best val NLL (nats)	-0.61 ± 0.59	-3.02 $\pm$ 1.61
best val ADE (m)	1.003 $\pm$ 0.032	0.986 $\pm$ 0.028

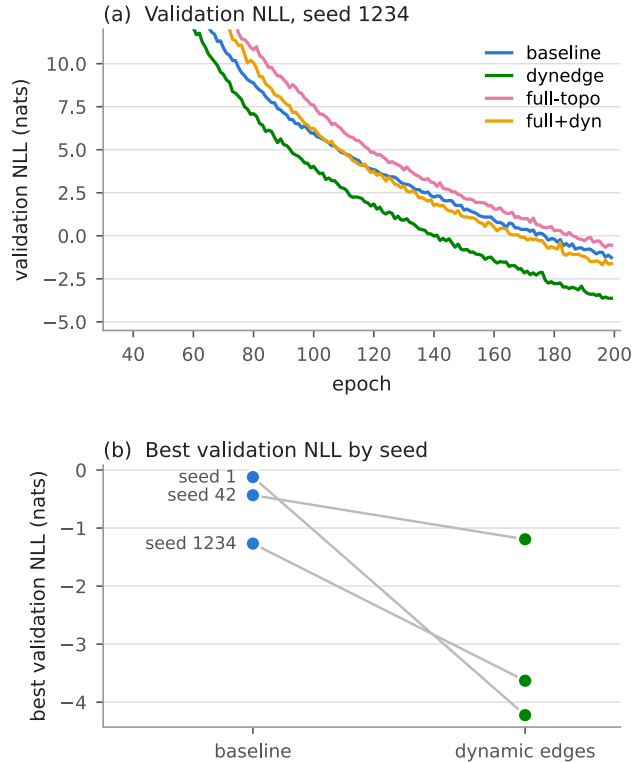


FIG. 1. (a) Validation NLL on inD (seed 1234) for the four decoder variants; curves shown after the objective-schedule transition. (b) Best validation NLL per seed: dynamic edges improve calibration in 3 of 3 seeds (mean -2.41 nats).

Removing the leak. Table II crosses topology (ground-truth vs. presence-agnostic) with edge features (none vs. dynamic) at seed 1234. Replacing the ground-truth future topology with the full graph costs at most 0.044 m ADE and 0.143 m FDE—at the edge of the ± 0.03 –0.12 seed band—so the published point

metrics of [1] were essentially honest despite the oracle (H3 confirmed); the full-topology run even attains the best validation FDE of all runs (2.756 m). Calibration, however, pays for the fix: best val NLL degrades from -1.27 to -0.67 without edge features. Adding dynamic edges recovers most of it (-1.69) and helps in *both* topology regimes ($-1.27 \rightarrow -3.63$ with the oracle; $-0.67 \rightarrow -1.69$ without), consistent with the phantom-agent mechanism H2: once presence is no longer oracle-filtered, only the distance kernel can silence departed agents.

TABLE II—Topology $\times$ edge features (seed 1234). FULL+DYN is the deployable configuration: no ground-truth future input.

Decoder	ADE	FDE	ANLL	val NLL
GT topo, no feat. ($=[1]$)	1.006	2.835	-0.108	-1.27
GT topo + dyn. edges	1.036	2.946	-0.197	-3.63
full topo, no feat.	1.048	2.968	-0.089	-0.67
full topo + dyn. edges	1.050	2.978	-0.131	-1.69

Secondary ablations. Two single-seed ablations bound other free choices of [1] (Table III). The mixture count $M=8$ is locally optimal on inD—both halving and doubling it hurt every metric—answering the natural “sweep M ” critique with a negative result. The integrator, left unspecified in the paper text, is load-bearing: replacing RK4 with forward Euler in (3) degrades ADE by $+51\%$ and FDE by $+37\%$, with validation error *diverging* after ≈ 50 epochs; the stiff-rollout accuracy of the motion model is a real dependency, not an implementation detail.

TABLE III—Design ablations of [1] on inD (seed 1234, baseline decoder).

Variant	ADE	FDE	ANLL	MR
$M=8$, RK4 (default)	1.006	2.835	-0.108	0.424
$M=4$	1.096	3.073	$+0.130$	0.435
$M=16$	1.067	3.023	-0.005	0.422
forward Euler	1.516	3.879	$+1.007$	0.594

Reproducibility. Both modifications ship as flags whose default reproduces the original model bit-for-bit: `--dynamic-edges` and `--full-edges`. Because the flags change the parameter set (the decoder gains an edge channel and a bandwidth), checkpoint filenames are tagged (DE/FE) so that baseline and variant weights can never cross-load silently. The full topology is generated from the sample’s node count at data-loading time rather than read from disk, adding no I/O and requiring no re-preprocessing of existing datasets. Inference overhead is one pairwise-distance computation per decoder step—negligible next to the GNN itself; the $\approx 25\%$ overhead quoted in Sec. IV-A is training-time. All runs, per-seed metric histories, and trained weights accompany the code.

VI. DISCUSSION: DID IT WORK?

The hypotheses were stated before the multi-seed runs; all three held. *H1 held:* the improvement concentrates where the mechanism predicts—in the likelihood. The gain is large on validation NLL (2.4 nats, 3/3 seeds) and clear on test ANLL, and qualitative inspection shows scene-adaptive uncertainty ellipses where the baseline’s grow monotonically. *H2 held:* without the presence oracle, dynamic edges improve validation NLL from -0.67 to -1.69 nats—more than undoing the loss from leak removal, ending better calibrated than the leaky

baseline itself (-1.27). *H3 held*: point accuracy neither improved nor degraded beyond seed noise—we explicitly do not claim an ADE/FDE win, and single-seed deltas below ≈ 0.07 m ADE on this benchmark should not be interpreted at all.

In what sense is the modified model *better*? First, it is better calibrated: for downstream planning under uncertainty, the covariances—not the point estimates—are the interface, and (5) improves materially at $\approx 25\%$ training overhead and negligible inference cost. Second, it is *deployable*: the `FULL+DYN` configuration consumes no ground-truth future information, closing the gap between benchmark protocol and a causal system at a cost (≤ 0.04 m ADE) that is within seed noise. Third, the analysis itself corrects the record on what MTP-GO’s numbers mean: they were obtained with a future-presence oracle, and we show the oracle was calibration-relevant but not accuracy-relevant.

When to adopt, and how far it travels. For any consumer of the predicted distribution—planners that integrate collision probability, risk-aware behavior selection, conformal wrappers—the calibrated variant dominates: same means, materially better likelihood, no future inputs. For leaderboard-style point-accuracy comparison it is neutral, and we would not advertise it there. The mechanism, however, is not MTP-GO-specific: the argument in Sec. IV-C uses only two structural facts—that a rollout decoder does message passing over an interaction graph, and that its uncertainty parameters are produced by that same message passing. Any graph-based encoder–decoder with covariance propagation of the form (4) exposes the same lever, and the same two-line audit (does the decoder see future topology? does it see any edge features?) applies verbatim to other reference implementations.

Limitations. Three seeds support direction-consistency, not formal significance ($p \approx 0.13$); the NLL seed variance is high (one seed is an outlier in both variants). Results are for one intersection dataset; highway scenes (highD), where relative geometry changes by ≈ 150 m per horizon, may respond differently on point accuracy—the mechanism predicts a larger effect there. The distance recomputation uses the mixture-pooled position, a single-point summary of a multimodal belief; per-mode graphs are a natural extension.

VII. CONCLUSION

Reading MTP-GO’s decoder closely revealed that its interaction graph is part oracle (ground-truth future presence)

and part inert (no edge features). Two minimal changes—recomputing distances from the model’s own rolling predictions through the existing learnable kernel, and a presence-agnostic topology—yield a decoder that is measurably better calibrated (≈ 2.4 nats validation NLL, consistent across seeds), equally accurate, and free of future information at inference. The broader lesson is methodological: in encoder–decoder forecasting models, the decoder’s input contract deserves the same scrutiny as the loss and the architecture, because that is where benchmark protocol and deployment reality quietly diverge.

Two extensions are immediate. Per-mode interaction graphs—computing (8) from each mixture component’s own position rather than the pooled mean—would let different maneuvers imply different interaction patterns, at M times the edge cost. And the highway setting is the natural stress test for H3: with ≈ 150 m of travel per 5 s horizon, the encoder’s frozen distance summary decays far faster than at an intersection, so dynamic edges may move displacement metrics there, not only the likelihood. Both are measurable with the released flags and require no new machinery.

REFERENCES

- [1] T. Westny, J. Oskarsson, B. Olofsson, and E. Frisk, “MTP-GO: Graph-based probabilistic multi-agent trajectory prediction with neural ODEs,” *IEEE Trans. Intelligent Vehicles*, vol. 8, no. 9, pp. 4223–4236, 2023.
- [2] T. Westny, J. Oskarsson, B. Olofsson, and E. Frisk, “Evaluation of differentially constrained motion models for graph-based trajectory prediction,” in *Proc. IEEE Intelligent Vehicles Symp. (IV)*, 2023.
- [3] J. Bock, R. Krajewski, T. Moers, S. Runde, L. Vater, and L. Eckstein, “The inD dataset: A drone dataset of naturalistic road user trajectories at German intersections,” in *Proc. IEEE Intelligent Vehicles Symp. (IV)*, 2020.
- [4] R. T. Q. Chen, Y. Rubanova, J. Bettencourt, and D. Duvenaud, “Neural ordinary differential equations,” in *Adv. Neural Information Processing Systems (NeurIPS)*, 2018.
- [5] C. Morris, M. Ritzert, M. Fey, W. L. Hamilton, J. E. Lenssen, G. Rattan, and M. Grohe, “Weisfeiler and Leman go neural: Higher-order graph neural networks,” in *Proc. AAAI Conf. Artificial Intelligence*, 2019.
- [6] S. Bengio, O. Vinyals, N. Jaitly, and N. Shazeer, “Scheduled sampling for sequence prediction with recurrent neural networks,” in *Adv. Neural Information Processing Systems (NeurIPS)*, 2015.
- [7] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, “On calibration of modern neural networks,” in *Proc. Int. Conf. Machine Learning (ICML)*, 2017.